```
Name:Siripuram Nithya Shree
Batch:03
Lab Assignment:13

!pip install -q scikit-learn pandas matplotlib

import os
import re
import csv
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn

from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, ConfusionMatrixDisplay
)
from torch.utils.data import Dataset, DataLoader

# 1. Set random seed and device
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

# 2. Load the CSV dataset
file_path = "/content/IMDB Dataset.csv"

if not os.path.exists(file_path):
    raise FileNotFoundError(
        "Upload IMDB Dataset.csv to Google Colab /content/ first."
    )

print("File size (MB):", round(os.path.getsize(file_path) / (1024 * 1024), 2))

# Use the CSV parser and skip malformed records if necessary.
# Standard IMDb CSV has two columns: review and sentiment.
try:
    df = pd.read_csv(
        file_path,
        engine="c",
        encoding="utf-8",
        encoding_errors="replace",
        on_bad_lines="skip",
        quoting=csv.QUOTE_MINIMAL
    )
except Exception as e:
    print("Standard CSV parsing failed:", e)
    print("Trying the Python parser...")
    df = pd.read_csv(
        file_path,
        engine="python",
        encoding="utf-8",
```

```
        encoding_errors="replace",
        on_bad_lines="skip"
    )

df.columns = df.columns.str.strip().str.lower()

print("Columns:", df.columns.tolist())
print("Loaded dataset shape:", df.shape)

if not {"review", "sentiment"}.issubset(df.columns):
    raise ValueError(
        "Expected columns 'review' and 'sentiment'. "
        "Please verify that you uploaded the correct IMDb CSV."
    )

# 3. Clean dataset and encode labels
df = df[["review", "sentiment"]].dropna().copy()

df["review"] = df["review"].astype(str)
df["sentiment"] = df["sentiment"].astype(str).str.strip().str.lower()

df = df[df["sentiment"].isin(["positive", "negative"])].copy()

df["label"] = df["sentiment"].map({
    "negative": 0,
    "positive": 1
})

df = df.drop_duplicates(subset=["review"]).reset_index(drop=True)

if len(df) == 0:
    raise ValueError("No valid reviews found. Check the CSV format.")

print("Clean dataset shape:", df.shape)
print(df["sentiment"].value_counts())

# 4. Split data into training and testing sets
train_df, test_df = train_test_split(
    df,
    test_size=0.2,
    random_state=SEED,
    stratify=df["label"]
)

train_df = train_df.reset_index(drop=True)
test_df = test_df.reset_index(drop=True)

print("Training samples:", len(train_df))
print("Testing samples:", len(test_df))

# 5. Tokenization and vocabulary creation
MAX_VOCAB = 20000
MAX_LEN = 200

def tokenize(text):
    return re.findall(r"\b\w+\b", text.lower())

word_counts = Counter()

for review in train_df["review"]:
    word_counts.update(tokenize(review))

vocab = {"<PAD>": 0, "<UNK>": 1}
```

```
for word, _ in word_counts.most_common(MAX_VOCAB - 2):
    if word not in vocab:
        vocab[word] = len(vocab)

print("Vocabulary size:", len(vocab))

def encode(text):
    tokens = tokenize(text)
    ids = [vocab.get(word, 1) for word in tokens[:MAX_LEN]]
    ids += [0] * (MAX_LEN - len(ids))
    return ids

# 6. Create PyTorch Dataset
class IMDbDataset(Dataset):
    def __init__(self, dataframe):
        self.reviews = dataframe["review"].tolist()
        self.labels = dataframe["label"].tolist()

    def __len__(self):
        return len(self.reviews)

    def __getitem__(self, idx):
        x = torch.tensor(encode(self.reviews[idx]), dtype=torch.long)
        y = torch.tensor(self.labels[idx], dtype=torch.float32)
        return x, y

BATCH_SIZE = 64

train_loader = DataLoader(
    IMDbDataset(train_df),
    batch_size=BATCH_SIZE,
    shuffle=True
)

test_loader = DataLoader(
    IMDbDataset(test_df),
    batch_size=BATCH_SIZE,
    shuffle=False
)

print("DataLoaders created successfully.")

# 7. Define RNN, LSTM and GRU
class SentimentModel(nn.Module):
    def __init__(self, model_type, vocab_size,
                 embedding_dim=64, hidden_dim=64):
        super().__init__()

        self.embedding = nn.Embedding(
            vocab_size, embedding_dim, padding_idx=0
        )

        if model_type == "RNN":
            self.rnn = nn.RNN(
                embedding_dim, hidden_dim, batch_first=True
            )
        elif model_type == "LSTM":
            self.rnn = nn.LSTM(
                embedding_dim, hidden_dim, batch_first=True
            )
        elif model_type == "GRU":
            self.rnn = nn.GRU(
                embedding_dim, hidden_dim, batch_first=True
```

```
            )
        else:
            raise ValueError("Model type must be RNN, LSTM or GRU.")

        self.fc = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        x = self.embedding(x)

        if isinstance(self.rnn, nn.LSTM):
            _, (hidden, _) = self.rnn(x)
        else:
            _, hidden = self.rnn(x)

        return self.fc(hidden[-1]).squeeze(1)

# 8. Training function
EPOCHS = 3
LEARNING_RATE = 0.001

def train_model(model_type):
    model = SentimentModel(
        model_type, len(vocab),
        embedding_dim=64, hidden_dim=64
    ).to(device)

    criterion = nn.BCEWithLogitsLoss()
    optimizer = torch.optim.Adam(
        model.parameters(), lr=LEARNING_RATE
    )

    history = {"loss": [], "accuracy": []}

    for epoch in range(EPOCHS):
        model.train()
        total_loss = 0.0
        correct = 0
        total = 0

        for x, y in train_loader:
            x, y = x.to(device), y.to(device)

            optimizer.zero_grad()
            logits = model(x)
            loss = criterion(logits, y)
            loss.backward()
            optimizer.step()

            total_loss += loss.item() * y.size(0)
            predictions = (logits >= 0).float()
            correct += (predictions == y).sum().item()
            total += y.size(0)

        epoch_loss = total_loss / total
        epoch_accuracy = correct / total

        history["loss"].append(epoch_loss)
        history["accuracy"].append(epoch_accuracy)

        print(
            f"{model_type} | Epoch {epoch + 1}/{EPOCHS} | "
            f"Loss: {epoch_loss:.4f} | "
            f"Accuracy: {epoch_accuracy:.4f}"
```

```
        )

    return model, history

# 9. Evaluation function
def evaluate_model(model):
    model.eval()
    actual, predicted = [], []

    with torch.no_grad():
        for x, y in test_loader:
            logits = model(x.to(device))
            predictions = (logits >= 0).long().cpu().numpy()

            actual.extend(y.long().numpy())
            predicted.extend(predictions)

    metrics = {
        "Accuracy": accuracy_score(actual, predicted),
        "Precision": precision_score(actual, predicted, zero_division=0),
        "Recall": recall_score(actual, predicted, zero_division=0),
        "F1-score": f1_score(actual, predicted, zero_division=0)
    }

    cm = confusion_matrix(actual, predicted)
    return metrics, cm

# 10. Train and evaluate all models
models = {}
histories = {}
results = {}
confusion_matrices = {}

for model_type in ["RNN", "LSTM", "GRU"]:
    print("\nTraining", model_type)

    model, history = train_model(model_type)
    metrics, cm = evaluate_model(model)

    models[model_type] = model
    histories[model_type] = history
    results[model_type] = metrics
    confusion_matrices[model_type] = cm

    print("Test results:")
    for metric, value in metrics.items():
        print(f"{metric}: {value:.4f}")

# 11. Plot training loss curves
plt.figure(figsize=(8, 5))

for model_type, history in histories.items():
    plt.plot(
        range(1, EPOCHS + 1),
        history["loss"],
        marker="o",
        label=model_type
    )

plt.xlabel("Epoch")
plt.ylabel("Training Loss")
plt.title("Training Loss: RNN vs LSTM vs GRU")
plt.legend()
```

```
plt.grid(True)
plt.show()

# 12. Compare evaluation metrics
metrics_df = pd.DataFrame(results).T
print("\nPerformance comparison:")
print(metrics_df.round(4))

metrics_df.plot(kind="bar", figsize=(10, 5), ylim=(0, 1))
plt.title("RNN vs LSTM vs GRU Performance")
plt.ylabel("Score")
plt.xlabel("Model")
plt.xticks(rotation=0)
plt.grid(axis="y")
plt.tight_layout()
plt.show()

# 13. Plot confusion matrices
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, model_type in zip(axes, ["RNN", "LSTM", "GRU"]):
    display = ConfusionMatrixDisplay(
        confusion_matrix=confusion_matrices[model_type],
        display_labels=["Negative", "Positive"]
    )
    display.plot(ax=ax, colorbar=False)
    ax.set_title(model_type)

plt.tight_layout()
plt.show()

# 14. Predict sentiment for a new review
def predict_sentiment(review, model):
    model.eval()

    x = torch.tensor(
        encode(review), dtype=torch.long
    ).unsqueeze(0).to(device)

    with torch.no_grad():
        probability = torch.sigmoid(model(x)).item()

    sentiment = "Positive" if probability >= 0.5 else "Negative"
    return sentiment, probability

sample_review = (
    "The movie was excellent, the acting was wonderful "
    "and I really enjoyed it."
)

print("\nSample review:", sample_review)

for model_type, model in models.items():
    sentiment, probability = predict_sentiment(sample_review, model)
    print(
        f"{model_type}: {sentiment}, "
        f"positive probability = {probability:.4f}"
    )

# 15. Find the best model based on test F1-score
best_model = max(results, key=lambda name: results[name]["F1-score"])

print("\nBest model:", best_model)
print("Best model test metrics:", results[best_model])
```

```
print("Best model test metrics:", results[best_model])
```